LEGS: Fine-Tuning Teleop-Free VLAs for Humanoid Loco-manipulation in an Embodied Gaussian Splatting World

Hojune Kim **Timothy Chen** **Jiankai Sun** **Lars W. Osterberg**
Qianzhong Chen **Ke Wang** **Mac Schwager**
Stanford University

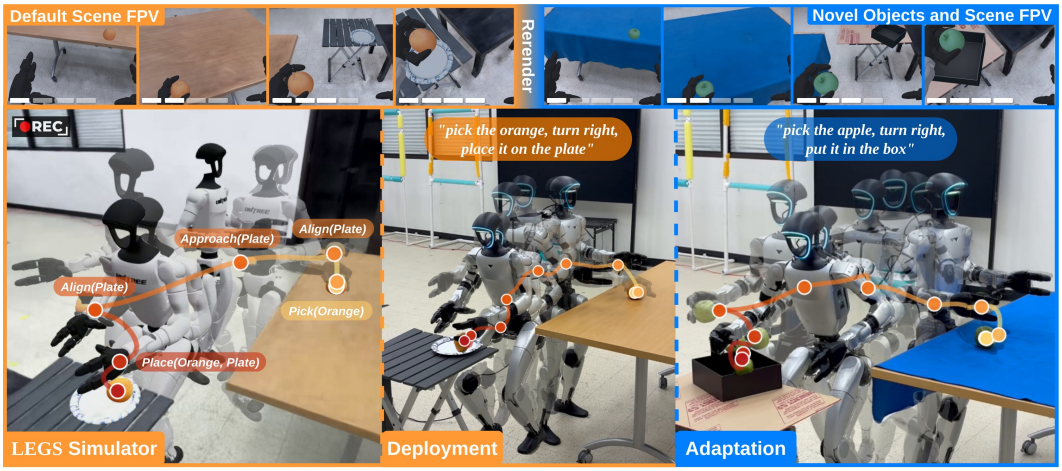


Figure 1: **L**oco-manipulation via **E**mbodied **G**aussian **S**platting (LEGS) combines the physical fidelity of MuJoCo, the visual fidelity of 3D Gaussian Splatting, and single-image object reconstruction from SAM3D to generate inexpensive humanoid training data without any real-robot data collection. Vision-Language-Action (VLA) models fine-tuned on LEGS data transfer zero-shot to the real robot and outperform those trained on teleoperation or mesh-only simulation.

Abstract: Training vision-language-action (VLA) policies for humanoid loco-manipulation is constrained by the high cost and complexity of collecting human teleoperation demonstrations. VLA policies fine-tuned in simulators have so far transferred poorly to humanoid loco-manipulation tasks. We present LEGS (**L**oco-manipulation via **E**mbodied **G**aussian **S**platting), a hybrid simulator that composites a mesh foreground (robot, objects, props) over a photorealistic 3D Gaussian Splatting (3DGS) background reconstructed from a handheld scene capture. LEGS uses a procedural motion-primitive generator to synthesize labeled demonstrations at scale without human teleoperation, and a two-stage color calibration that aligns renders to the deployment camera. On a Unitree G1 humanoid robot, across three pick-and-place tasks of increasing whole-body difficulty and three VLA backbones (ψ_0, $\pi_{0.5}$, GR00T N1.6), a policy trained purely on LEGS data matches or exceeds one trained on human teleoperation demos on every experiment. It also outperforms a mesh-only baseline trained on the same motions without the calibrated 3DGS frontend. Because motion is recorded independently of appearance, the same demonstrations can be re-rendered for a target scene and objects at over 15$\times$ lower cost than teleoperation. When both the objects and the scene change, a policy trained on LEGS-AUG data re-rendered for that target maintains task success, while the teleop baseline fails entirely. Code and videos are available at https://hojjunekim.github.io/legs/.

Keywords: Humanoid, Sim-to-real Transfer, Robot Data Generation, Robot Simulator, Loco-Manipulation, Vision Language Action Model

correspondence to hojune@stanford.edu

10th Conference on Robot Learning (CoRL 2026), Austin TX, USA.

1 Introduction

The humanoid form factor is uniquely suited to operate alongside humans: shared morphology allows direct use of human-built environments, tools, and infrastructure without retrofitting. Recent vision-language-action (VLA) foundation models [1, 2, 3, 4] show that large-scale pre-training unlocks impressive zero-shot manipulation, but adapting them to a specific humanoid platform still requires fine-tuning demonstrations covering both manipulation and locomotion skills. *Teleoperation* [5, 6, 7] binds data to operator labor and robot uptime, while *human-demo retargeting* [8, 9, 10] removes the robot from the loop, but still requires per-scene egocentric capture. Any appearance shift at deployment requires human labor for each new episode, and operator and hardware fatigue make extended collection sessions increasingly costly (Table 1).

Collecting data in simulation mitigates human labor, but conventional simulators introduce a *visual gap*: mesh-based rendering produces images whose distribution diverges from real camera footage, and even strong mesh-synthetic pipelines built on SAM3D [11] reconstructions suffer from material-shading mismatch, simplified lighting, and aliasing that bias the visual encoder. 3D Gaussian Splatting (3DGS) [12], in contrast, is trained to minimize rendering error against *real* photographs from the target scene, substantially reducing the visual sim-to-real gap.

We propose LEGS, a photorealistic simulator that generates humanoid loco-manipulation training data without any teleoperation. LEGS builds on established real-to-sim components (MuJoCo physics, 3DGS, and SAM3D object reconstruction); our contribution is their composition into a teleop-free data pipeline for humanoid VLA fine-tuning, together with an extensive real-robot study of when such data transfers. LEGS renders a dynamic mesh foreground (robot, manipulable objects, props) over a static 3DGS background reconstructed from a handheld scene capture, and composites the two through a deterministic two-stage color calibration that aligns the render to the deployment camera. Because the foreground motion is recorded as a command stream independent of the rendering inputs, a procedural generator synthesizes labeled demonstrations at scale, and the same motion dataset re-renders under new backgrounds and object meshes at GPU cost alone. We evaluate LEGS on a Unitree G1 across three pick-and-place tasks of increasing whole-body difficulty and three VLA backbones (ψ_0, $\pi_{0.5}$, GR00T N1.6), benchmarked against human teleoperation and a mesh-only synthetic baseline. Our contributions are as follows:

- **A teleop-free data pipeline for humanoid VLA fine-tuning.** LEGS generates labeled loco-manipulation data entirely in simulation, with no human teleoperation or per-episode human intervention; task semantics are instead specified once per task as a plan over shared motion primitives (Appendix G.2). A policy fine-tuned purely on LEGS data matches or exceeds one fine-tuned on human teleoperation across all three tasks and backbones, including a budget-matched 50-demonstration comparison and a long-horizon task on which teleop fails entirely. Against a mesh-only baseline that shares the same motion data but lacks the calibrated 3DGS frontend, LEGS wins on every experiment at all dataset sizes.
- **A parallelizable re-rendering workflow for low-labor adaptation to target appearances.** Because motion is decoupled from appearance, LEGS re-renders one recorded motion dataset under swapped backgrounds and object meshes. For each additional appearance condition with an existing scene capture, LEGS replaces $>$1.5 hr of human teleop labor (Table 1) with $\sim$0.1 hr of GPU time on consumer-grade hardware. The resulting LEGS-AUG dataset, re-rendered for the target objects and scene, retains task success under that combined shift, while teleop fails.
- **Extensive real-robot evaluation across three SOTA VLAs and four data conditions (1,110 trials, plus 240 matched-camera trials).** The same findings reproduce across ψ_0, $\pi_{0.5}$, and GR00T N1.6 fine-tuned from their public checkpoints, suggesting they are a property of the data pipeline rather than any single VLA. Because LEGS produces a general-purpose humanoid loco-manipulation demonstration dataset, the same recorded-and-augmented data can fine-tune other VLA backbones that accept the same command interface.

Table 1: **Data sources for humanoid loco-manipulation policy training.** Teleop and LEGS costs measured on Task 3 (long-horizon, 50 episodes); new condition cost is the marginal cost of producing 50 episodes for one additional appearance condition with an existing scene capture. LEGS costs cover data execution and rendering only; they exclude the one-time framework engineering and the per-task setup (Appendix G.2). Human-demo cost is extrapolated from the per-episode timing ratio reported in [13] on the Unitree G1.

	No on-robot data collection	No per-episode human	Re-renderable to new scenes	Initial cost (hr)	New condition cost (hr)
Teleop [5, 6, 14]	✗	✗	✗	1.5	>1.5
Human demo [10, 13, 15]	✓	✗	✗	∼1.0	>1.0
LEGS (ours)	✓	✓	✓	**0.5**	**0.1**

2 Related Work

Humanoid loco-manipulation and VLA foundation models. Producing the whole-body motion that positions a humanoid for loco-manipulation typically relies on shadowing or wearable-device teleoperation [5, 6] or decoupling locomotion from manipulation [14], all of which require on-robot data collection. Robot-free human-demonstration systems [10, 13, 15] avoid teleoperation but remain expensive to scale due to per-scene human capture. In parallel, vision-language-action foundation models have advanced rapidly [2, 3, 16]. We instead generate motion procedurally in simulation and fine-tune three pretrained VLAs on the resulting data.

Teleoperation-free humanoid loco-manipulation. Concurrent work explores humanoid loco-manipulation without on-robot teleoperation through several paradigms: sim-augmented behavior cloning from a single seed demonstration [17], distillation of a privileged RL teacher into a vision-based student [18, 19], and reference-tracking RL from optimization-derived trajectories [20]. Each removes the operator but still requires per-task specification: a hand-collected seed, a hand-engineered reward, or a hand-designed trajectory. LEGS also requires per-task specification, in the form of a task plan over shared motion primitives (Appendix G.2), but involves no human teleoperation or per-episode human intervention—no seed demonstration, no human video, no per-task reward—and fine-tunes pretrained VLA backbones rather than training from scratch. A privileged RL teacher [18] and a procedural task plan are thus alternative ways to encode a task; a head-to-head comparison is left to future work.

Real-to-sim-to-real and photorealistic simulation. Building simulators from real-world captures is an established route to robot learning. RialTo [21] robustifies imitation policies learned from a few real demonstrations with RL in scanned digital twins, PolaRiS [22] turns short video scans into simulated environments for scalable policy evaluation, DRAWER [23] reconstructs articulated interactive scenes, and PhysTwin [24] recovers deformable-object physics from video. Earlier sim-to-real work instead randomizes non-photorealistic renderers [25, 26], and mesh reconstruction via SAM3D [11] scales synthetic data but inherits the mesh-rendering gap; we use it as our non-3DGS baseline. 3D Gaussian Splatting [12] has been used as a simulator backbone for fixed-base manipulation [27, 28], high-throughput RL on locomotion and tabletop tasks [29, 30], visuomotor drone navigation [31, 32, 33, 34], and aerial manipulation [35]. Whereas these works target manipulation, locomotion, navigation, or evaluation, LEGS applies real-to-sim to teleop-free demonstration generation for fine-tuning pretrained VLAs on humanoid loco-manipulation, where decoupling recorded motion from appearance lets one motion dataset be re-rendered for new target appearances.

3 The LEGS Framework

The framework is organized around two decouplings. *Physics from rendering*: a MuJoCo [36] backend running a low-level whole-body controller resolves dynamics on mesh geometry, while a separate rendering frontend produces image observations. *Static background from dynamic foreground*: the

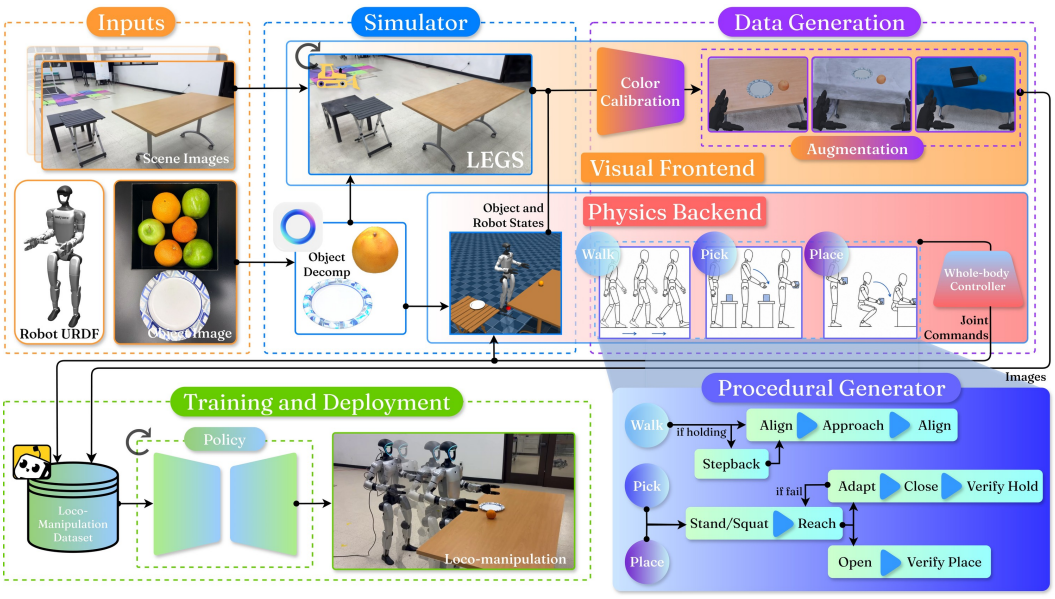


Figure 2: **The LEGS pipeline.** A scene video and object photo are reconstructed into a 3DGS background and SAM3D meshes, which feed the LEGS simulator. The simulator decouples a **visual frontend** (3DGS and mesh compositor with color calibration) from a **physics backend** (MuJoCo with the low-level whole-body controller). A procedural generator produces labeled demonstrations, re-rendered under scene and object augmentations, and used to fine-tune a VLA backbone (ψ_0, $\pi_{0.5}$, GR00T N1.6) for real-robot deployment.

background is a fixed 3DGS field rendered for appearance only, the foreground is rasterized meshes that move under physics, and layers are composited per frame by a depth test. Motion recorded against the foreground meshes is therefore independent of scene appearance, so a single motion dataset can be re-rendered under swapped backgrounds and meshes (Figure 2).

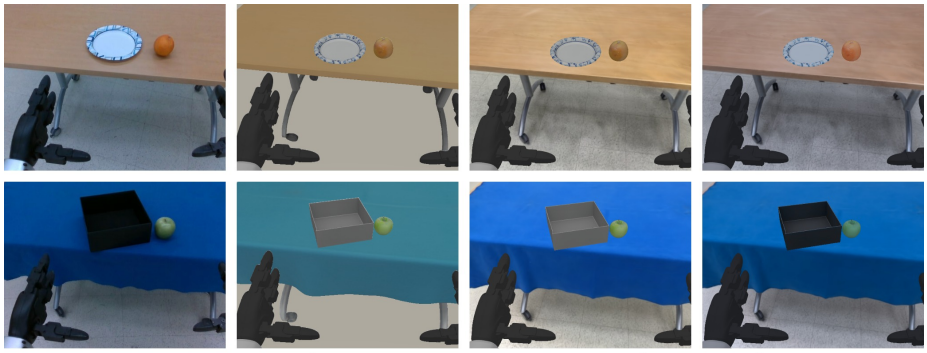
3.1 Visual Frontend

Background and foreground. Each scene is captured with a ∼1–2 minute handheld video and reconstructed into a 3DGS field via COLMAP [37] poses followed by 3DGS optimization (Appendix A). Because 3DGS minimizes rendering error against real photographs, novel views are photorealistic, reducing the visual sim-to-real gap relative to mesh rendering. The foreground meshes come from the robot URDF and SAM3D [11] reconstructions of objects and props from a single handheld image. At each frame the foreground is rasterized and depth-composited with the background.

Color calibration. The composite mixes two color spaces, the scanning device's ISP (3DGS) and SAM3D's regressed albedo (meshes), which must be mapped to a third, the on-robot deployment camera. We disable auto-exposure and auto-white-balance on both cameras so the mapping is stationary. We close the gap in two stages:

$$c_d \approx M\,[R(c_m) \sqcup c_b],$$

where c_m is a mesh vertex color, c_b a background 3DGS pixel, c_d the deployment-camera pixel, $R(\cdot)$ a per-mesh diagonal scale in linear RGB matching each mesh's rendered mean to the handheld scan inside its SAM2 [38] mask, $\sqcup$ depth-test compositing, and $M \in \mathbb{R}^{3\times3}$ a global color-correction matrix fit by least squares from a 24-patch ColorChecker photographed by both cameras. As Figure 3 shows, the per-mesh scale R corrects SAM3D's mis-regressed albedo—the box that SAM3D renders as washed-out grey is restored to black—while the global matrix M shifts the 3DGS background (table and floor) toward the deployment camera, together bringing the composite close to the real on-robot image. Derivations are in Appendix C.



(a) GT (b) SAM3D (c) LEGS w/o calib (d) LEGS w/ calib

Figure 3: **Egocentric views of the same task moment.** Columns: (a) real on-robot image, (b) SAM3D mesh-only baseline, (c, d) raw and color calibrated LEGS renders. The second row shows the *same recorded motion* re-rendered under swapped object and scene meshes.

3.2 Physics Backend and Controller Interface

The physics backend resolves dynamics on mesh geometry in MuJoCo at 500 Hz, separately from the rendering pipeline. The dynamic foreground reuses the SAM3D object meshes and URDF robot model from Section 3.1, with object collision geometry decomposed via CoACD [39]. The static background has its own collision mesh, derived from a SAM3D reconstruction aligned to the 3DGS field.

We use SONIC [40] as the low-level whole-body controller, an RL-trained policy that handles whole-body coordination from a high-level command interface. SONIC accepts an 18-D command shared between simulation and the real robot: a 6-D wrist SE(3) pose plus a continuous grip scalar per arm (14-D upper-body), concatenated with a 4-D base command (v_x, v_y, ω_z, h). The same SONIC binary runs in simulation and on the robot, so deployment requires no changes to the deployed policy.

3.3 Procedural Episode Generation

LEGS leverages the privileged information (robot, object poses) inherent to a simulator to procedurally generate episodes, further reducing human labor. LEGS composes parametrized motion primitives whose arguments are scene-level (target pose, object identity, arm choice) rather than joint-level, which lets a single task plan produce a different valid trajectory under each randomized initial condition. Each pick-and-place task decomposes into three high-level motions (`Walk`, `Pick`, `Place`), each composed of lower-level primitives (Figure 2). Each motion ends with a verification check, and only verified successful episodes are saved (85–94% of generated episodes across tasks). Randomization ranges, the full primitive protocol, and the split between shared and per-task specification are in Appendices D and G.

The recorded 18-D command stream is independent of the rendering inputs, so the resulting dataset can be re-rendered under swapped backgrounds and meshes at GPU cost only, with no new motion collection (Section 4.4).

4 Experiments

We evaluate LEGS on three pick-and-place tasks of increasing whole-body difficulty, fine-tuning three pretrained VLA backbones (ψ_0, $\pi_{0.5}$, GR00T N1.6) on each of three data conditions, including human teleoperation and a mesh-only synthetic baseline.

4.1 Experimental Setup

Tasks and protocol. The three tasks (Figure 4) are all pick-and-place on the Unitree G1. Across all tasks the robot picks an orange and places it on a white plate sitting on a wood-grain table. Task 1

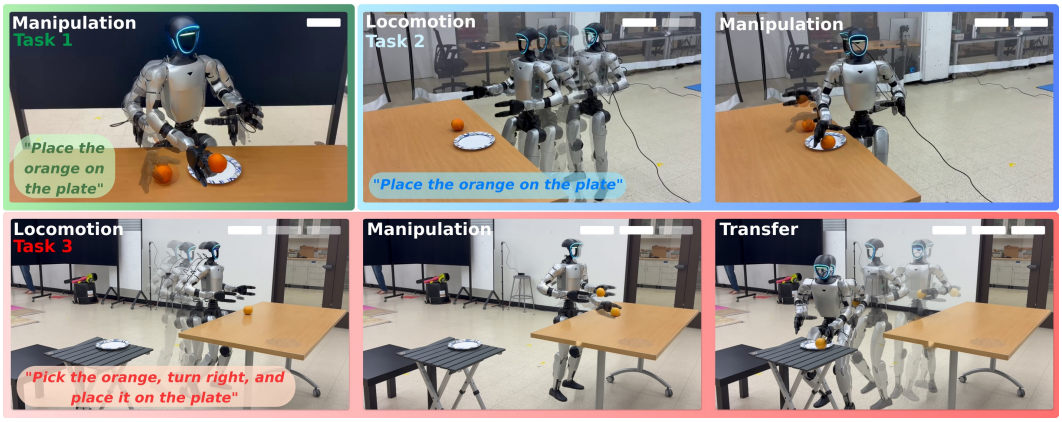


Figure 4: **Three pick-and-place tasks of increasing whole-body difficulty. Task 1 (manipulation-only)**: pick and place the orange on the plate without locomotion. **Task 2 (loco-manipulation, easy)**: walk to the table, then pick and place. **Task 3 (loco-manipulation, hard)**: walk, pick, turn, walk to a low table, squat, and place.

and Task 2 share the language prompt *"place the orange on the plate"*, and Task 3 uses *"pick the orange, turn right, and place it on the plate"*. Figure 8 (Appendix H) shows the full Task 3 trajectory annotated with its subtask stages and motion primitives, alongside the egocentric views for each data condition. A trial can fail at any subtask, so we report end task success rate (TSR) in the main text and stage-wise cumulative success in Appendix B. The robot has a 29-DoF body and 7-DoF Dex3 hand per side, with a single head-mounted Intel RealSense D435 streaming RGB at 30 Hz. We run $R = 10$ trials per data condition, backbone, and task, with initial conditions drawn independently per trial: the orange and plate positions are perturbed by ± 5 cm and the robot heading by $\pm 10^\circ$, plus the robot base position by ± 10 cm on the loco-manipulation tasks. On the long-horizon Task 3 we extend this to 20 trials per cell in Appendix I.

Data conditions. We compare three data conditions. **Teleop (50)** collects 50 real demonstrations per task, recorded by a non-expert operator via SONIC's VR 3-point teleoperation interface [40]. **SAM3D (200)** is a mesh-only synthetic baseline identical to LEGS (200) except that the 3DGS background is replaced by SAM3D meshes and color calibration is disabled (Appendix E). **LEGS (200)** is our main dataset, produced by the procedural generator of Section 3.3 and rendered as mesh foreground composited over the 3DGS background with color calibration enabled. Because color calibration assumes a fixed camera ISP, LEGS policies are deployed with fixed exposure and white balance, while Teleop and SAM3D use the camera's default automatic mode; Appendix I removes this confound by re-evaluating LEGS under the automatic mode. We also report **LEGS (50)**, the first 50 episodes of LEGS (200), for a budget-matched comparison against teleop. The appearance-adaptation experiment of Section 4.4 runs on the strongest in-distribution backbone per task from Table 2, ψ_0 for Tasks 1–2 and GR00T N1.6 for Task 3.

Foundation-model fine-tuning. We fine-tune three VLAs from their released checkpoints (ψ_0 [16], $\pi_{0.5}$ [2], GR00T N1.6 [3]), all sharing SONIC as the controller and the 18-D command of Section 3.2. Within each backbone the fine-tuning recipe is fixed across data conditions, so within-backbone comparisons isolate the data source. Recipes differ across backbones (full fine-tuning for $\pi_{0.5}$, action-head-only for the others). Full per-backbone details are in Appendix J.

4.2 Performance Across Three VLA Backbones

LEGS matches or exceeds teleop. LEGS (200) is best or tied on all nine (backbone, task) experiments of Table 2, and the gap over Teleop (50) widens with task difficulty: modest on Task 1, larger on Task 2, and decisive on Task 3 where Teleop (50) collapses to 0/10 across all backbones

Table 2: **End task success out of 10 trials, per data condition, backbone, and task.** LEGS (200) is our main dataset. LEGS (50) is the first 50 episodes of LEGS (200), for a budget-matched comparison against the 50-demo teleop baseline. Best per (backbone, task) in **bold**. Stage-wise cumulative successes for the same experiments are shown in Appendix B.

	Task 1			**Task 2**			**Task 3**		
Data condition	ψ_0	$\pi_{0.5}$	GR00T N1.6	ψ_0	$\pi_{0.5}$	GR00T N1.6	ψ_0	$\pi_{0.5}$	GR00T N1.6
Teleop (50)	8/10	2/10	4/10	6/10	0/10	2/10	0/10	0/10	0/10
SAM3D (200)	7/10	1/10	6/10	4/10	3/10	5/10	1/10	0/10	3/10
LEGS (50)	8/10	5/10	**9/10**	8/10	4/10	7/10	3/10	1/10	5/10
LEGS (200)	**10/10**	**6/10**	**9/10**	**9/10**	**5/10**	**8/10**	**5/10**	**2/10**	**6/10**

while LEGS (200) achieves 2–6/10. The ranking holds across all three backbones despite different fine-tuning recipes, suggesting that the gain is a property of the data pipeline, not any one backbone. Individual 10-trial cells are underpowered, so we draw conclusions from aggregates: across the nine experiments, LEGS (200) reaches 60/90 (66.7%, Wilson 95% CI 56.4–75.5) versus 22/90 for Teleop (50) (24.4%, 16.7–34.2) and 30/90 for SAM3D (200) (33.3%, 24.5–43.6).

The gain is not a budget artifact. The budget-matched LEGS (50) condition matches or beats Teleop (50) on every experiment, including Task 3 where teleop produces zero successes, indicating that the improvement is not explained by LEGS's larger 200-demonstration budget alone. The further gap from LEGS (50) to LEGS (200) is consistent with standard imitation-learning data scaling. Appendix B shows that this gap appears at the `pick` and `place` stages, the precise close-range manipulation stages where extra demonstrations help most, while the methods stay comparable at `walk`.

4.3 Effect of the Rendering Frontend

The SAM3D (200) condition shares the LEGS motion data and pipeline but replaces the 3DGS background with SAM3D meshes and disables color calibration. SAM3D (200) lies strictly below LEGS (200) on every experiment of Table 2: it averages 33% end task success against 67% for LEGS (200), a 33-point mean drop that roughly halves the success rate. SAM3D retains success comparable to LEGS at `walk`, but diverges from LEGS during `pick` and `place` on every task and backbone. Because the two conditions differ in background representation, color calibration, and deployment camera mode, this comparison establishes a frontend-level effect of calibrated 3DGS rendering over an untuned mesh renderer, not the contribution of 3DGS alone. To remove the camera confound, Appendix I re-evaluates the Task 3 LEGS checkpoints with the camera in the same automatic mode as the baselines (n=20 per cell): LEGS (200) changes from 23/60 to 18/60 (p=0.41), still ahead of SAM3D (200) at 6/60 ($p < 0.01$, stratified by backbone). Separating the 3DGS background from color calibration requires retraining and is left to future work.

4.4 Low-Labor Adaptation to Target Scene Appearance

LEGS's appearance-from-rendering decoupling (Section 3.3) predicts that adapting to a new scene's appearance should require only re-rendering recorded motion, not collecting new motion. We test this on four appearance conditions, varying objects and scene independently (Figure 5), measuring both per-condition collection cost and real-robot TSR.

Appearance and prompt swap. The *default* condition uses the orange-and-plate setup and prompts of Section 4.1. The *scene* condition swaps only the table for a blue table and keeps the default prompt. The *objects* and *objects + scene* conditions further swap the orange for an apple and the plate for a box, with the prompt updated to *"put the apple in the box"* (Tasks 1–2) or *"pick the apple, turn right, and put it in the box"* (Task 3). For LEGS-AUG and SAM3D-AUG, training episodes are re-rendered with the new meshes and re-labeled with the new prompt (composition in Appendix D).

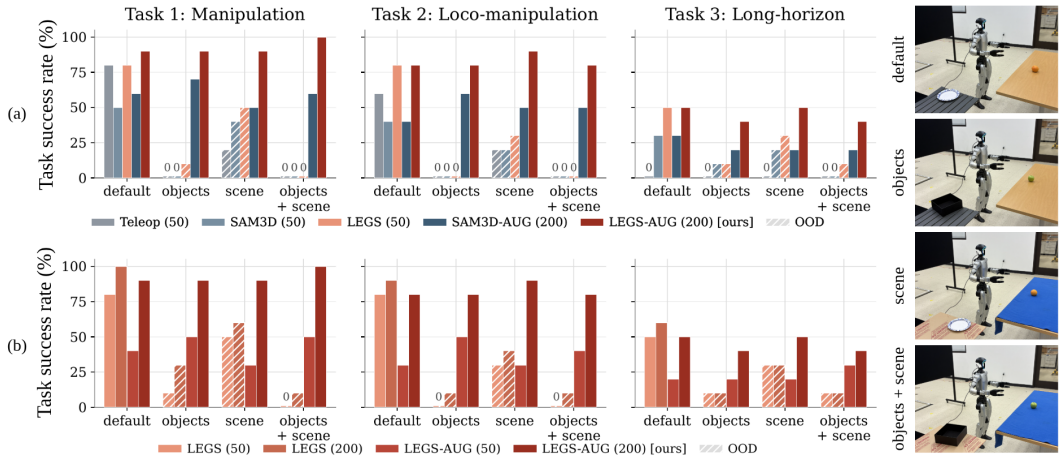


Figure 5: **Real-robot TSR (%) under appearance shift.** Variations cover *default*, *objects* (apple and box), *scene* (blue table), and *objects* + *scene*, shown as third-person views on the right. Hatched bars are out-of-distribution for the training set, solid bars are in-distribution. Tasks 1–2 are evaluated on ψ_0, Task 3 on GR00T N1.6. **(a)** Comparison across data sources at the 50-episode budget against re-rendered SAM3D-AUG (200) and LEGS-AUG (200). **(b)** Within-LEGS ablation isolating dataset size from re-rendering.

Because the augmented training set contains the exact evaluation scene, object meshes, and prompts, this experiment measures target-specific adaptation, not generalization to unseen assets. Teleop, SAM3D (50), and LEGS (50) are trained on the default data only and see the shifted visuals and prompt out of distribution at test time. Teleop cannot be re-rendered, and the un-augmented synthetic baselines are matched to teleop's training budget for fairness.

Cost. Producing 50 Task 3 demonstrations takes 1.5 operator-hours for teleop and roughly 0.5 GPU-hours for LEGS on a single RTX 4090, with no operator involvement. Each additional appearance condition then costs more than 1.5 operator-hours for teleop, which must re-collect motion in the new scene, versus roughly 0.1 GPU-hours for LEGS (and $\sim$0.2 for the SAM3D baseline), which only re-render the existing motion (Appendix F); a new scene additionally needs a 1–2 minute handheld video and unattended 3DGS reconstruction. Adapting to a target scene therefore shifts from an operator-bound to a compute-bound problem, and the compute is parallelizable across GPUs.

Re-rendering enables adaptation; the calibrated 3DGS frontend improves it. Without re-rendering, all three 50-episode baselines (Teleop (50), SAM3D (50), and LEGS (50)) collapse to 0–1/10 on *objects* and *objects* + *scene* across all three tasks, confirming that the joint appearance-and-prompt shift is genuine and that no method generalizes to it from default data alone. With re-rendering, both SAM3D-AUG (200) and LEGS-AUG (200) retain success on every shifted appearance condition (Figure 5a), with LEGS-AUG consistently ahead: on the hardest *objects* + *scene* shift it reaches 100%, 80%, and 40% on Tasks 1–3, versus 60%, 50%, and 20% for SAM3D-AUG. Teleop cannot participate in this gain because a physically captured dataset cannot be re-rendered.

Augmentation beats scale under object-and-prompt shift. Figure 5(b) isolates dataset size from augmentation within LEGS: on the *objects* and *objects* + *scene* shifts, re-rendering into the new object and prompt distribution (LEGS-AUG (50)) beats not only LEGS (50) but also the fourfold-larger LEGS (200) (on *objects* + *scene* it reaches 50%, 40%, and 30% on Tasks 1–3 while LEGS (200) stays at 10% across all tasks) because the default-only policy never sees the new object or prompt. On the milder *scene*-only shift, the larger default dataset is comparable or better, so re-rendering pays off specifically when the shift changes what the policy must recognize, not merely the background.

5 Limitations

LEGS removes teleoperation but not task engineering: each task needs a plan over shared primitives, and new task families such as articulated-object interaction or tool use would need new primitives (insertion would also need force control). The one-time cost of building the framework is not included in our cost comparisons. All three evaluated tasks are structured pick-and-place, and our attribution of the gain over the mesh baseline is to the calibrated 3DGS frontend as a whole; a factorial ablation of background and calibration, and comparisons against a tuned, photometrically randomized mesh renderer, target-condition teleoperation, or a privileged RL teacher, are left to future work. Each cell uses 10 trials (20 on Task 3 in Appendix I) and a single training seed, so per-cell differences are not individually conclusive, and we do not report a taxonomy of falls, collisions, or safety interventions during evaluation. LEGS-AUG adapts to specified target appearances rather than generalizing to unseen ones. Our color calibration disables auto-exposure and auto-white-balance to keep the mapping stationary, so the trained policy is sensitive to lighting changes outside the distribution captured during the original 3DGS scan. Adapting to a new scene still requires a 1–2 minute handheld video, which is cheaper than teleop re-collection but not free. The 3DGS reconstruction degrades on highly reflective or transparent surfaces and assumes a static background, so scenes with moving humans or other dynamic content would require dynamic-3DGS methods that we do not implement. Our procedural generator produces feasible motion but does not optimize for dynamic efficiency, unlike approaches based on trajectory optimization or MPC. The appearance-adaptation suite varies only small graspable objects of similar size and geometry, so transfer to substantially different object categories is not tested. We evaluate on a single robot platform (Unitree G1), camera (Intel RealSense D435), and whole-body controller (SONIC); generalization across humanoid hardware is not tested. Finally, scanning real deployment spaces may capture private information and should be done with appropriate consent.

6 Conclusion

We presented LEGS, a hybrid simulator that composites a mesh foreground over a photorealistic 3DGS background, together with a procedural generator, producing labeled humanoid loco-manipulation demonstrations without any teleoperation, seed demonstration, or human video. Across three pick-and-place tasks and three pretrained VLA backbones on a Unitree G1, LEGS data matches or exceeds human teleoperation on every experiment, outperforms a mesh-only synthetic baseline without the calibrated 3DGS frontend, and adapts to specified object-and-scene appearance targets at a fraction of the operator cost. For this task family, calibrated 3DGS rendering and procedural motion generation can replace humanoid teleoperation at the data-collection step.

Acknowledgments

This work was supported in part by ONR grant N00014-23-1-2354 and NSF grant 2342246. Timothy Chen is supported by a NASA NSTGRO fellowship. We are grateful for this support.

References

[1] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[2] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[3] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.

[4] R. Firoozi, J. Tucker, S. Tian, A. Majumdar, J. Sun, W. Liu, Y. Zhu, S. Song, A. Kapoor, K. Hausman, et al. Foundation models in robotics: Applications, challenges, and the future. *The International Journal of Robotics Research*, 44(5):701–739, 2025.

[5] T. He, Z. Luo, X. He, W. Xiao, C. Zhang, W. Zhang, K. Kitani, C. Liu, and G. Shi. OmniH2O: Universal and dexterous human-to-humanoid whole-body teleoperation and learning. *arXiv preprint arXiv:2406.08858*, 2024.

[6] Y. Ze, Z. Chen, J. P. Araújo, Z.-a. Cao, X. B. Peng, J. Wu, and C. K. Liu. TWIST: Teleoperated whole-body imitation system. *arXiv preprint arXiv:2505.02833*, 2025.

[7] Q. Ben, F. Jia, J. Zeng, J. Dong, D. Lin, and J. Pang. HOMIE: Humanoid loco-manipulation with isomorphic exoskeleton cockpit. *arXiv preprint arXiv:2502.13013*, 2025.

[8] C. Chi, Z. Xu, C. Pan, E. Cousineau, B. Burchfiel, S. Feng, R. Tedrake, and S. Song. Universal manipulation interface: In-the-wild robot teaching without in-the-wild robots. *arXiv preprint arXiv:2402.10329*, 2024.

[9] Z. Fu, Q. Zhao, Q. Wu, G. Wetzstein, and C. Finn. HumanPlus: Humanoid shadowing and imitation from humans. *arXiv preprint arXiv:2406.10454*, 2024.

[10] H. Jiang, J. Chen, Q. Bu, L. Chen, M. Shi, Y. Zhang, D. Li, C. Suo, C. Wang, Z. Peng, et al. WholeBodyVLA: Towards unified latent VLA for whole-body loco-manipulation control. *arXiv preprint arXiv:2512.11047*, 2025.

[11] X. Chen, F.-J. Chu, P. Gleize, K. J. Liang, A. Sax, H. Tang, W. Wang, M. Guo, T. Hardin, X. Li, et al. SAM 3D: 3Dfy anything in images. *arXiv preprint arXiv:2511.16624*, 2025.

[12] B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis, et al. 3D Gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4):139–1, 2023.

[13] M. Shi, S. Peng, J. Chen, H. Jiang, Y. Li, D. Huang, P. Luo, H. Li, and L. Chen. EgoHumanoid: Unlocking in-the-wild loco-manipulation with robot-free egocentric demonstration. *arXiv preprint arXiv:2602.10106*, 2026.

[14] C. Lu, X. Cheng, J. Li, S. Yang, M. Ji, C. Yuan, G. Yang, S. Yi, and X. Wang. Mobile-TeleVision: Predictive motion priors for humanoid whole-body control. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5364–5371. IEEE, 2025.

[15] R. Nai, B. Zheng, J. Zhao, H. Zhu, S. Dai, Z. Chen, Y. Hu, Y. Hu, T. Zhang, C. Wen, et al. Humanoid manipulation interface: Humanoid whole-body manipulation from robot-free demonstrations. *arXiv preprint arXiv:2602.06643*, 2026.

[16] S. Wei, H. Jing, B. Li, Z. Zhao, J. Mao, Z. Ni, S. He, J. Liu, X. Liu, K. Kang, S. Zang, W. Yuan, M. Pavone, D. Huang, and Y. Wang. Ψ_0: An open foundation model towards universal humanoid loco-manipulation. In *Robotics: Science and Systems*, 2026. To Appear. Preprint arXiv:2603.12263.

[17] Y. Fu, F. Xie, C. Xu, J. Xiong, H. Yuan, and Z. Lu. DemoHLM: From one demonstration to generalizable humanoid loco-manipulation. *arXiv preprint arXiv:2510.11258*, 2025.

[18] T. He, Z. Wang, H. Xue, Q. Ben, Z. Luo, W. Xiao, Y. Yuan, X. Da, F. Castañeda, S. Sastry, et al. VIRAL: Visual sim-to-real at scale for humanoid loco-manipulation. *arXiv preprint arXiv:2511.15200*, 2025.

[19] H. Xue, T. He, Z. Wang, Q. Ben, W. Xiao, Z. Luo, X. Da, F. Castañeda, G. Shi, S. Sastry, et al. Opening the sim-to-real door for humanoid pixel-to-action policy transfer. *arXiv preprint arXiv:2512.01061*, 2025.

[20] F. Liu, Z. Gu, Y. Cai, Z. Zhou, H. Jung, J. Jang, S. Zhao, S. Ha, Y. Chen, D. Xu, et al. Opt2Skill: Imitating dynamically-feasible whole-body trajectories for versatile humanoid loco-manipulation. *IEEE Robotics and Automation Letters*, 2025.

[21] M. Torne, A. Simeonov, Z. Li, A. Chan, T. Chen, A. Gupta, and P. Agrawal. Reconciling reality through simulation: A real-to-sim-to-real approach for robust manipulation. In *Proceedings of Robotics: Science and Systems*, 2024. doi:10.15607/RSS.2024.XX.015.

[22] A. Jain, M. Zhang, K. Arora, W. Chen, M. Torne, M. Z. Irshad, S. Zakharov, Y. Wang, S. Levine, C. Finn, W.-C. Ma, D. Shah, A. Gupta, and K. Pertsch. PolaRiS: Scalable real-to-sim evaluations for generalist robot policies. *arXiv preprint arXiv:2512.16881*, 2025.

[23] H. Xia, E. Su, M. Memmel, A. Jain, R. Yu, N. Mbiziwo-Tiapo, A. Farhadi, A. Gupta, S. Wang, and W.-C. Ma. DRAWER: Digital reconstruction and articulation with environment realism. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 21771–21782, 2025.

[24] H. Jiang, H.-Y. Hsu, K. Zhang, H.-N. Yu, S. Wang, and Y. Li. PhysTwin: Physics-informed reconstruction and simulation of deformable objects from videos. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 7219–7230, 2025.

[25] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pages 23–30. IEEE, 2017.

[26] J. Sun, A. Curtis, Y. You, Y. Xu, M. Koehle, Q. Chen, S. Huang, L. Guibas, S. Chitta, M. Schwager, et al. Arch: Hierarchical hybrid learning for long-horizon contact-rich robotic assembly. *CoRL*, 2024.

[27] G. Jiang, H. Chang, R.-Z. Qiu, Y. Liang, M. Ji, J. Zhu, Z. Dong, X. Zou, and X. Wang. GSWorld: Closed-loop photo-realistic simulation suite for robotic manipulation. *arXiv preprint arXiv:2510.20813*, 2025.

[28] O. Shorinwa, J. Tucker, A. Smith, A. Swann, T. Chen, R. Firoozi, M. D. Kennedy, and M. Schwager. Splat-MOVER: Multi-stage, open-vocabulary robotic manipulation via editable Gaussian splatting. In *8th Annual Conference on Robot Learning*, 2024.

[29] A. Escontrela, J. Kerr, A. Allshire, J. Frey, R. Duan, C. Sferrazza, and P. Abbeel. GaussGym: An open-source real-to-sim framework for learning locomotion from pixels. *arXiv preprint arXiv:2510.15352*, 2025.

[30] Y. Jia, H. Zhang, Z. Zhang, J. Wu, M. Yu, Z. Wang, D. Jiang, Z. Li, C. Cao, Z. Yu, et al. GS-Playground: A high-throughput photorealistic simulator for vision-informed robot learning. *arXiv preprint arXiv:2604.25459*, 2026.

[31] J. Low, M. Adang, J. Yu, K. Nagami, and M. Schwager. SOUS VIDE: Cooking visual drone navigation policies in a Gaussian splatting vacuum. *IEEE Robotics and Automation Letters*, 2025.

[32] M. Adang, J. Low, O. Shorinwa, and M. Schwager. SINGER: An onboard generalist vision-language navigation policy for drones. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2026.

[33] Q. Chen, J. Sun, N. Gao, J. Low, T. Chen, and M. Schwager. GRaD-Nav: Efficiently learning visual drone navigation with Gaussian radiance fields and differentiable dynamics. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 7941–7948. IEEE, 2025.

[34] Q. Chen, N. Gao, S. Huang, J. Low, T. Chen, J. Sun, and M. Schwager. GRaD-Nav++: Vision-language model enabled visual drone navigation with Gaussian radiance fields and differentiable dynamics. *IEEE Robotics and Automation Letters*, 11(2):1418–1425, 2025.

[35] J. Tucker, D. Liu, A. Swann, A. Ren, J. Yu, J. Sun, B. Kim, L. McGranahan, Q. Vuong, and M. Schwager. π, but make it fly: Physics-guided transfer of VLA models to aerial manipulation, 2026. URL https://arxiv.org/abs/2603.25038.

[36] E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pages 5026–5033. IEEE, 2012.

[37] J. L. Schonberger and J.-M. Frahm. Structure-from-motion revisited. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4104–4113, 2016.

[38] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, et al. SAM 2: Segment anything in images and videos. In *International Conference on Learning Representations*, volume 2025, pages 28085–28128, 2025.

[39] X. Wei, M. Liu, Z. Ling, and H. Su. Approximate convex decomposition for 3D meshes with collision-aware concavity and tree search. *ACM Transactions on Graphics (TOG)*, 41(4):1–18, 2022.

[40] Z. Luo, Y. Yuan, T. Wang, C. Li, S. Chen, F. Castaneda, Z.-A. Cao, J. Li, D. Minor, Q. Ben, et al. SONIC: Supersizing motion tracking for natural humanoid whole-body control. *arXiv preprint arXiv:2511.07820*, 2025.

[41] B. Yi, C. M. Kim, J. Kerr, G. Wu, R. Feng, A. Zhang, J. Kulhanek, H. Choi, Y. Ma, M. Tancik, et al. Viser: Imperative, web-based 3D visualization in Python. *arXiv preprint arXiv:2507.22885*, 2025.

A Simulator Implementation Details

A.1 Reconstruction Pipeline

Background reconstruction: (1) iPhone video capture (handheld, $\sim$1–2 minute sweep of the room), (2) COLMAP sparse reconstruction for camera pose estimation, (3) 3DGS training through nerfstudio (30K iterations, single workstation GPU, $\sim$10–15 min). Alternatively, one can use the many image-to-3DGS apps (e.g. Polycam).

For the background collision mesh used by physics, we reconstruct a mesh with SAM3D [11] and align it to the 3DGS field via the shared COLMAP poses. Some image-to-3DGS apps (e.g. Kiri Engine) instead export a mesh in the same frame as the 3DGS reconstruction, skipping this alignment.

Reconstruction robustness. Because object stability depends on the reconstructed support surfaces, we checked their geometry and the resulting contact behavior. Across ten independent SAM3D reconstructions of each Task 3 table, the solid tabletops are planar to within 1°. In 400 simulated placements of the task objects at task poses, no object slid or fell.

A.2 Front-End / Back-End Interface

As noted in Section 3.2, the simulator front-end and the SONIC controller back-end use identical interfaces in simulation and on the real robot. Concretely, they communicate over ROS/DDS pub-sub: the front-end publishes the 18-dim command stream and receives proprioceptive state, and the same DDS topic names are reused across deploy targets. For interactive visualization and scene inspection during episode generation, the front-end uses viser [41].

B Stage-Wise Cumulative Success

Each task in Section 4.1 is a sequence of subtasks, so a trial can fail at any stage, and the stage at which a method fails is itself diagnostic. Figure 6 reports the per-stage cumulative success underlying the end-TSR numbers of Table 2. Across most cells the methods are comparable at the `walk` stage but separate during `pick` and `place`, the camera regime in which the egocentric view moves close to the object and rendering fidelity dominates. Teleop typically collapses toward zero at these stages, while SAM3D plateaus at a lower success level than LEGS; LEGS retains the highest success through the manipulation stages on every backbone.

C Color Calibration Pipeline

The composited observation mixes three color spaces: the iPhone ISP output (3DGS), SAM3D's regressed albedo (meshes), and the RealSense D435 ISP output (the deployment camera). To keep the mapping stationary, we disable auto-exposure and auto-white-balance on both cameras. This only applies to LEGS; SAM3D and teleop apply no color calibration and run the RealSense in its default auto-exposure/auto-white-balance mode. We report the deployment RealSense D435 settings (exposure 10 ms, white balance 3000 K), and the iPhone scan settings are locked but absorbed into the Stage-2 calibration. We then align the three spaces in two stages.

Stage 1: mesh-to-deployment albedo match. The SAM3D vertex colors approximate the input texture but are systematically muted, as the network regresses surface albedo rather than the ISP tone curve of the scanning device, and the mesh is rendered under physically based (PBR) lighting that further modulates the baked colors. A chart-based calibration does not apply, since no chart is visible in an arbitrary object photograph. We instead match the rendered object statistics to the source photograph directly. For each mesh we render an interior view under the deployment renderer's PBR lighting (directional headlight intensity 1.0, ambient 0.3) and take $\mu_S^{(c)}$, the per-channel mean of linear RGB over pixels with $\alpha > 200$. We pair it with $\mu_T^{(c)}$, the per-channel mean of linear RGB over

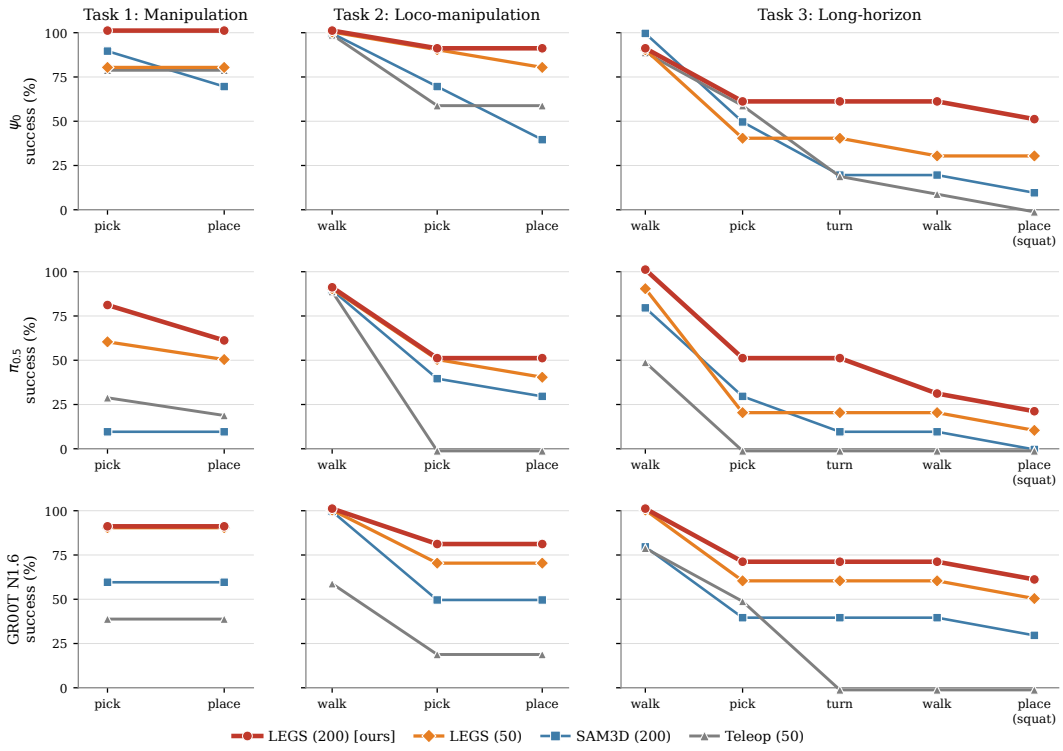


Figure 6: **Stage-wise cumulative success, faceted by task (columns) and backbone (rows).** Each panel shows four data conditions (LEGS (200) [ours], LEGS (50), SAM3D (200), Teleop (50)) tracing the percentage of 10 trials still successful through each subtask stage; the final stage equals overall TSR. Small per-condition y-offsets keep overlapping lines visible.

iPhone pixels inside the SAM2 mask. The diagonal scale

$$s^{(c)} \;=\; \mathrm{clip}\left(\frac{\mu_T^{(c)}}{\mu_S^{(c)}},\; 0.5,\; 3.0\right)$$

is multiplied into the mesh's linear vertex colors, which are then clipped to $[0, 1]$ and re-exported; geometry is unchanged. Operating diagonally in linear RGB rather than as a full 3×3 keeps the multiplier well-conditioned: a full-covariance fit needed entries above 6 to match the iPhone's wider intra-region variance, which clipped vertex colors and shifted hue. Restricting the target statistics to the masked region is essential, since background pixels would bias the match toward scene illumination rather than the object. Two conventions matter: the scale is fit against the PBR-lit render used at data-generation time, not an idealized unlit appearance; and since glTF 2.0 stores vertex colors as linear RGB, the calibrated colors are written in linear RGB so the renderer's gamma-encode recovers the intended sRGB appearance without double-encoding.

Stage 2: iPhone-to-RealSense color-correction matrix. With all assets in iPhone space, a single 3×3 matrix maps composited frames to RealSense space. We photograph a 24-patch color chart with both cameras under controlled illumination, extract the median sRGB value of each patch, and linearize via the sRGB EOTF. Stacking the linear RealSense and iPhone patch colors as $P, Q \in \mathbb{R}^{24\times3}$, the matrix is the least-squares solution

$$M^\star \;=\; \arg\min_M \|P - QM\|_F^2 \;=\; (Q^\top Q)^{-1} Q^\top P.$$

At render time, each composited frame is linearized, multiplied by $M^\star$, clipped to $[0, 1]$, and re-encoded to sRGB. Applying $M^\star$ at the composite level rather than per object keeps meshes and the 3DGS background spatially consistent with no boundary artifacts.

D Kinematic Randomization Parameters

Per-reset randomization ranges, sampled uniformly. "Object" ranges apply to every randomized object in the scene (the picked object and the placement target).

Quantity	Task 1 (manipulation)	Tasks 2–3 (loco-manip)
Robot x, y offset (cm)	0	± 10
Robot heading (deg)	± 10	± 10
Object x, y offset (cm)	± 5	± 5
Object yaw (deg)	± 180	± 180

For the appearance-condition evaluation (Section 4.4), the objects and tables are swapped for the target-appearance assets (orange and plate, apple and box, wood or blue table) and the episodes are re-rendered; no photometric augmentation is applied. LEGS-AUG (50) allocates 12/13/12/13 episodes to the *default*, *objects*, *scene*, and *objects + scene* conditions, and LEGS-AUG (200) uses 50 episodes per condition.

E SAM3D Synthetic Baseline Details

Reconstruction. We reconstruct each object (orange, plate, tables, and the swap assets used in Section 4.4) from a single handheld image using SAM3D [11] and extract textured meshes; the robot is loaded from its manufacturer URDF, as in the LEGS condition.

Renderer. A standard mesh rasterizer renders the entire scene (foreground objects plus floor, walls, and props), with no 3DGS background and color calibration disabled. We did *not* tune the renderer to defeat the visual gap; this matches a typical mesh-synthetic pipeline a practitioner would build.

Otherwise identical. The procedural episode generator, MuJoCo physics, CoACD collision decomposition, SONIC WBC backend, action and state spaces, per-reset kinematic randomization, and the 200-demo budget are all held identical to the LEGS condition, so differences are attributable to the rendering frontend (3DGS background and color calibration) and the deployment camera mode rather than data scale; Appendix I removes the camera-mode difference. No photometric augmentation is applied.

F Dataset-Collection Cost

Table 3: **Dataset-collection cost** (wall-clock hours, Task 3, 50 episodes each). *Initial* is the cost of the first 50-episode dataset; the right column is the marginal cost of one additional appearance condition.

Dataset	Initial cost (hr)	+ 1 appearance condition (hr)
Teleop (50)	1.5 (operator)	$>$1.5 (operator + hardware fatigue)
SAM3D (50)	0.5 (compute)	0.2 (re-render only)
LEGS (50)	0.5 (compute)	0.1 (re-render only)

Teleop (50) is one operator collecting 50 demonstrations of Task 3, the most expensive task at $\sim$30 s per episode. The synthetic conditions instead run the procedural generator at real-time physics for initial motion capture, then re-render the recorded motion on a single RTX 4090 with +20 CPU cores for each additional appearance condition. LEGS renders mesh foreground over a rasterized 3DGS background, while SAM3D re-shades the full mesh scene (foreground plus background props), making its re-render slightly more expensive.

The table makes two asymmetries visible. *(i) Initial dataset*: both synthetic conditions produce 50 demonstrations in 3$\times$ less wall-clock time than teleop and with zero operator hours, since the procedural generator runs headlessly and shares one motion dataset. *(ii) Per-condition scaling*: each

additional appearance condition costs only GPU time for the synthetic conditions ($\sim$0.1 hr for LEGS, $\sim$0.2 hr for SAM3D) but more than 1.5 hr of operator work for teleop, whose motion is bound to the capture scene and must be re-collected. The second asymmetry is what makes covering many appearance conditions practical in simulation and impractical for teleop; the LEGS-vs-SAM3D gap further shows the 3DGS background is cheaper to re-render than a full mesh scene.

G Data Collection Protocol

G.1 Teleoperation Setup

We use SONIC's VR whole-body teleoperation interface from GR00T-WholeBodyControl [40], operated in its three-point (`vr3pt`) tracking mode. The operator wears a PICO VR device: the headset and two hand controllers supply head and dual-wrist poses that SONIC retargets to the wrist targets, the controller triggers set the per-arm grip, and the joysticks set the 4-D base command (v_x, v_y, ω_z, h); Figure 7). SONIC consumes these inputs and produces the shared 18-D high-level command stream driving both the simulator and the real robot. The 50 real demonstrations per task were collected over multiple sessions; episodes with fall events or hardware safety triggers were discarded and re-collected to maintain a balanced 50-episode budget, and this re-collection is included in the operator-hour figure of Appendix F.

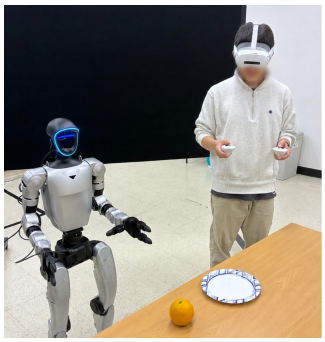


Figure 7: VR three-point teleoperation setup.

G.2 Primitive-Based Generation Protocol

Primitive composition. Each pick-and-place task decomposes into three high-level motions: `Walk`, `Pick`, and `Place`. `Walk` takes a target object and runs `Align`, `Approach`, then `Align` again to position the robot in front of the object; the final `Align` commands base yaw (ω_z) to refine the robot's heading toward the active arm—a pre-fixed grasping arm for `Pick` or the arm currently holding the object for `Place`—so the target falls into a better reach angle for that arm. A `Stepback` primitive runs first when `Walk` is invoked while holding an object. `Pick` and `Place` share a common opening, with `Stand/Squat` adjusting the pelvis height to the object height h_{obj} and `Reach` interpolating the end-effector toward an initial pose estimate of the target.

`Pick` then closes the loop with an `Adapt` primitive that corrects the gripper pose using the commanded-vs-actual wrist offset, a privileged simulator signal used only during data generation and not at deployment; on a closure-tolerance failure the arm retracts to a re-pose and re-runs `Reach` and `Adapt`. Each motion ends with a verification primitive—`VerifyHold` after `Pick` and `VerifyPlace` after `Place`—that confirms success before the episode is recorded.

Execution protocol. For each task, the scene config declares a `task_plan`, an ordered list of task primitives, and the manager executes the plan against the current scene state. Each primitive reports whether it is still running, has completed, or has failed; on failure the manager applies primitive-specific retries (e.g., re-attempting an `Approach` with a slightly offset waypoint) before declaring episode failure. Successful episodes are retained toward the 200-demo budget; episodes that fail verification or any primitive are discarded and not counted.

Action encoding and verification parameters. The 18-D command is ordered as per-arm blocks of [wrist 6-D pose, grip], left arm then right arm, followed by the base command (v_x, v_y, ω_z, h). Wrist poses are expressed relative to the pelvis, with rotations as Euler-XYZ angles, and commands are issued at 30 Hz. `VerifyHold` uses a 0.10 m object–wrist tolerance along z; `VerifyPlace` requires the object to lie within 0.10 m in xy and 0.06 m in z of the target surface. `Pick` allows two grasp-servo (`Adapt`) iterations and one outer retry.

Task specification and generation yield. LEGS removes teleoperation and per-episode human intervention, but each task is still specified by hand. Shared across all tasks and built once are the motion primitives, verification predicates, module-level tolerances and retry limits, the episode manager, the controller interface, and color calibration. Specified per scene or task are the scene geometry and object roles, the ordered task plan, the grasping arm and initial target poses, the language prompt, the randomization ranges, and per-object grasp parameters. This per-scene and per-task part takes under 15 operator-minutes, in addition to an unattended 3DGS reconstruction. Within the pick-and-place family we reuse the existing primitives and retune grasp parameters per object; a new task family would require new primitives. Building the framework is a one-time cost that is not repeated per task and is not included in Tables 1 and 3. Across Tasks 1–3, retaining 200 verified episodes took 213–235 generated attempts, a yield of 85–94%.

H Task 3 Trajectory and Per-Condition Egocentric Views

Figure 8 documents Task 3, our longest-horizon task, at two levels of granularity and across the three data conditions.

Stages and primitives (a). The top two rows show eight third-person keyframes from a GR00T N1.6 policy trained on LEGS data, each carrying two labels: a coarse *stage* in the top-left (red), which is exactly the stage used in the stage-wise success plot of Appendix B (`Walk`, `Pick`, `Turn`, `Walk`, `Squat`, `Place`), and the underlying *motion primitive(s)* in the bottom-right (white), which realize those stages and compose the `Walk`/`Pick`/`Place` high-level motions of Appendix G. Read together they trace the full task: `Walk` to the orange (`Align`/`Approach`) and `Pick` it (`Reach`/`Close`, then `Idle`/`Stepback`); `Turn` toward the low table (`Align`); a second `Walk` across to it (`Approach`/`Align`); and `Squat`/`Place` to lower the pelvis and release the orange (`Squat`/`Reach`, then `Open`).

Per-condition egocentric views (b–d). The lower rows show the egocentric views at the same eight stages. Panels (b) and (c) re-render the *same* recorded motion, so they are pose-aligned frame-by-frame and differ only in rendering frontend—LEGS composites mesh foreground over the 3DGS background with color calibration, while SAM3D renders the full scene (Appendix E); panel (d) is a representative teleoperated rollout. The contrast between (b) and (c) illustrates the close-range gap of Section 4.3: LEGS's 3DGS background renders the room photorealistically, whereas the SAM3D render is comparatively flat and uniform. Panel (d) is shown only as a reference for the real task; because teleop runs the RealSense with auto-exposure and auto-white-balance enabled while LEGS is deployed at a fixed ISP, (b) and (d) differ in exposure and color and are not directly comparable pixel-for-pixel.

I Matched-Camera Evaluation on Task 3

In Table 2, LEGS policies are deployed with fixed exposure and white balance, while Teleop and SAM3D policies use the RealSense's automatic mode. To remove this confound, we re-evaluate the same LEGS checkpoints with the camera in automatic mode, which also lies outside the regime in which the color-correction matrix was fitted. We use the long-horizon Task 3 and extend every cell to n=20 trials; for the conditions in Table 2, the first 10 trials are those reported there. Initial conditions are drawn independently per trial from the ranges of Appendix D.

Under matched automatic settings, LEGS still leads at both data budgets. With Cochran–Mantel–Haenszel tests stratified by backbone, LEGS (50) exceeds Teleop (50) (p=0.0008) and LEGS (200) exceeds SAM3D (200) (p=0.009). Holding the checkpoint fixed, moving LEGS (200) from fixed to automatic camera settings reduces success from 23/60 to 18/60, a difference that is not statistically distinguishable under the same stratified test (p=0.41). On $\pi_{0.5}$ every automatic-mode condition is at or below 1/20, a floor effect. Because LEGS and SAM3D still differ in both background representation and color calibration, this comparison establishes a frontend-level effect only.

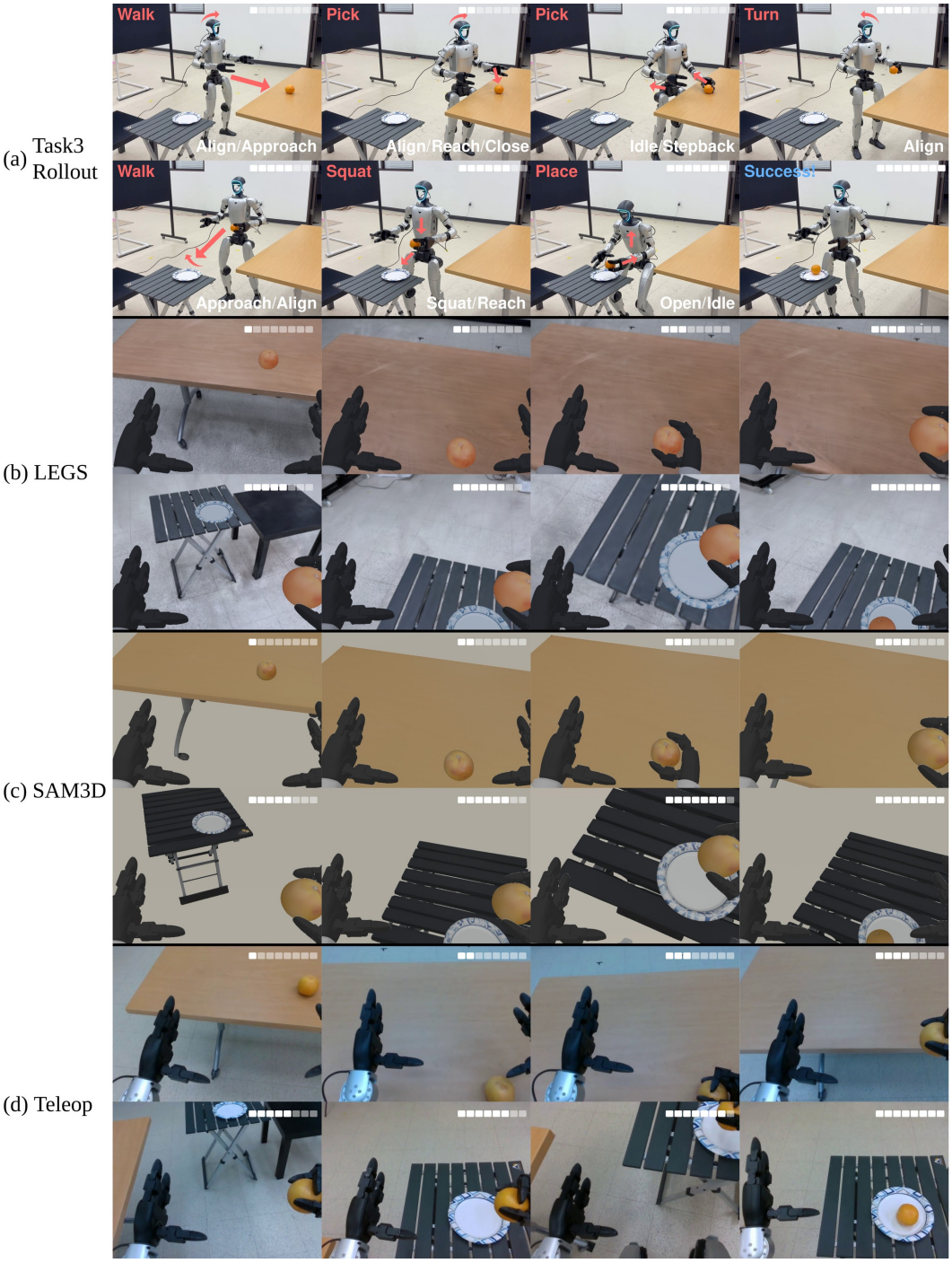


Figure 8: **Task 3 trajectory and per-condition egocentric observations. (a)** Eight third-person keyframes of a successful Task 3 rollout. Each frame is labeled with its *stage* (top-left, red), matching the stages of the stage-wise success plot (Appendix B), and the active *motion primitive(s)* (bottom-right, white), which compose the `Walk`/`Pick`/`Place` high-level motions of Appendix G. **(b–d)** Head-camera observations at the same eight keyframes for LEGS, the SAM3D mesh-only baseline, and human teleoperation. Panels (b, c) re-render one shared motion and are pose-aligned; (d) is a representative teleoperated rollout. Note (d) is captured with the RealSense auto-exposure/auto-white-balance enabled, whereas LEGS (b) is calibrated to the fixed deployment ISP, so the two are not a like-for-like color match.

Table 4: **Task 3 end-task success under matched and fixed camera settings,** n=20 **per cell.** "Auto" is the RealSense automatic exposure and white-balance mode used by the baselines; "fixed" is the LEGS deployment setting.

Data / deployment ISP	ψ_0	$\pi_{0.5}$	GR00T N1.6	All
Teleop (50) / auto	1/20	0/20	0/20	1/60
SAM3D (200) / auto	2/20	0/20	4/20	6/60
LEGS (50) / auto	5/20	1/20	8/20	14/60
LEGS (200) / auto	8/20	1/20	9/20	18/60
LEGS (50) / fixed	5/20	2/20	10/20	17/60
LEGS (200) / fixed	**9/20**	**3/20**	**11/20**	**23/60**

J Policy Training Details

All three backbones share SONIC as the whole-body controller (Section 3.2) and emit the same 18-dimensional command, so within-backbone comparisons isolate the data source. Each checkpoint is trained with a single seed, and LEGS (50) is a fixed first-50 prefix of LEGS (200), so we do not estimate seed or subset variance. They differ only in fine-tuning scope and the per-backbone hyperparameters of Table 5.

Table 5: **Per-backbone fine-tuning configuration.** All runs: bf16 mixed precision, 20,000 optimization steps, 4×H100 80GB GPUs, 18-D (EEF + base) action, peak learning rate 10^{-4} with a 1k-warmup cosine schedule, and flow-matching loss.

	ψ_0	$\pi_{0.5}$	GR00T N1.6
Fine-tuning scope	action head only	full (VLM + head)	action head only
Action chunk	30	16	16
Effective batch	256	128	24
Checkpoint cadence	every 2,500 steps	every 2,500 steps	every 10,000 steps
Initialization	ψ_0	DROID-pretrained $\pi_{0.5}$	GR00T-N1.6-3B